



Project Sentrrix: *Gases and Scents identification with MQs and BME688 based on Machine Learning*

Tenne Tian Rebecca Li, Rahul Prasanna

ENGINEERING
SUMMER ACADEMY at PENN



Penn
Engineering
UNIVERSITY of PENNSYLVANIA

TABLE OF CONTENTS



01 Introduction
Why this idea?

03 Our Product
What we did

02 Market Analysis
What else is there?

04 Conclusion
All in all...



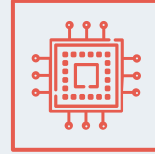
GOALS



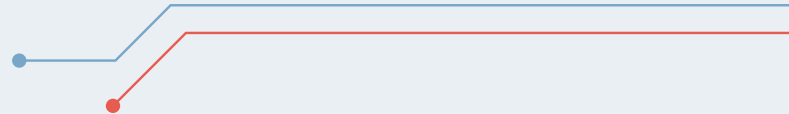
Classify and
Distinguish Between
Gases and Scents



Visual and Auditory
Output



Calibrate the Sensor for
Concentration Tracking





Why GAS and SCENT?

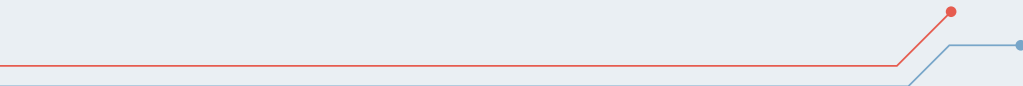


WHAT IF YOUR NOSE

Findings

In 2021, the global mortality rate due to unintentional carbon monoxide poisoning was 0·366 per 100 000 (95% uncertainty interval 0·276–0·415), with 28 900 deaths (21 700–32 800) and 1·18 million YLLs (0·886–1·35) across all ages. Nearly 70% of deaths occurred in males (20 100 [15 800–24 000]), and the 50–54-year age group had the largest number of deaths (2210 [1660–2590]). The highest mortality rate was in those aged 85 years or older with 1·96 deaths (1·38–2·32) per 100 000. Eastern Europe had the highest age-standardised mortality rate at 2·12 deaths (1·98–2·30) per 100 000. Globally, there was a 53·5% (46·2–63·7) decrease in the age-standardised mortality rate from 2000 to 2021, although this decline was not uniform across regions. The overall PAFs for occupational injuries and high alcohol use were 13·6% (11·9–16·0) and 3·5% (1·4–6·2), respectively.

Moberg, Madeline E, et al. "Global, Regional, and National Mortality due to Unintentional Carbon Monoxide Poisoning, 2000–2021: Results from the Global Burden of Disease Study 2021." *The Lancet Public Health*, vol. 8, no. 11, 1 Nov. 2023, pp. e839–e849, www.sciencedirect.com/science/article/pii/S2468266723001858?via%3Dihub, [https://doi.org/10.1016/S2468-2667\(23\)00185-8](https://doi.org/10.1016/S2468-2667(23)00185-8).



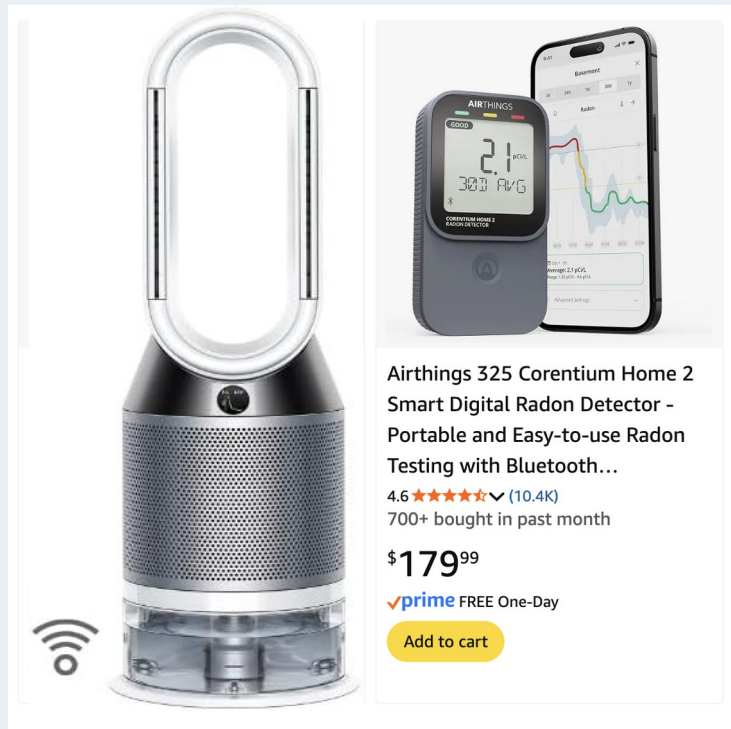
Market Analysis

What if your nose could detect danger, freshness, and your favorite scent—all at once? It can't, but with our AI-powered sensor system, it can.



Why Are we Competitive

- Cost Efficiency: ~\$30–\$50, offers the same VOC trained scents and has a high accuracy of reaction time, within the range of 20 seconds
- Duo Sensor: duo sensor system (MQ + BME688) provides robust verification for mixed gases and scents; BME688's AI potential enables unique applications like coffee aroma detection.
- Reverse-engineering: By detecting and profiling gas emissions, the system can map out scent and gas components



dyson pure cool



HOW WE DID IT



(it was with a lot of work)

BME688

VOC

Detects volatile organic compounds (VOCs) — used for air quality and smell detection.

Humidity

Measures ambient temperature

Gas Resistance

How the electrical resistance of the sensor's material changes when it interacts with a specific gas.

Pressure

Measures barometric pressure



MQ SENSORS

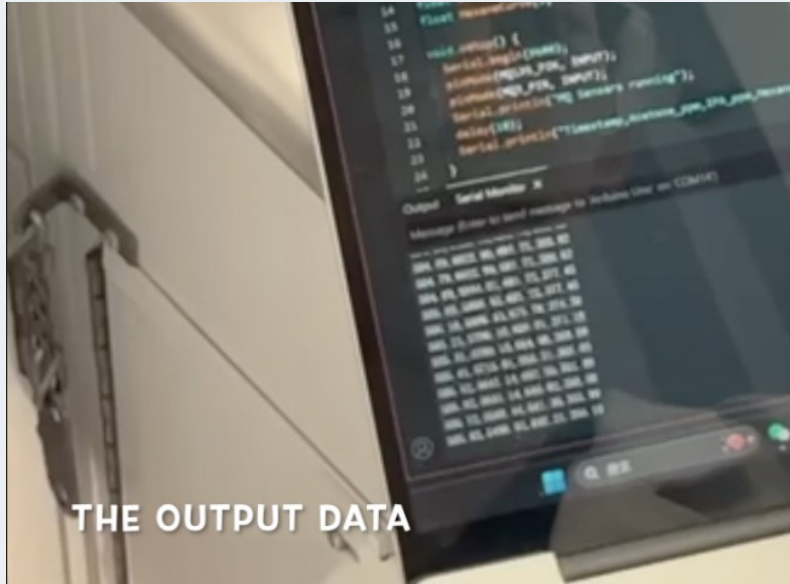


MQ-135 - Carbon Dioxide,
Alcohol, Acetone



MQ-3 - Methanol, Isopropyl
Alcohol,

MQ SENSOR DATA



Our output data from the MQs came out in terms of parts per million. Ppm is often used as a way to express very low concentration of a substance within a much larger substance. We used the serial plotter within the Arduino IDE to create a live graph of the concentration of gases.

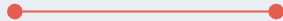


BME688 training



Code—Sensor Setup

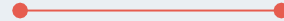
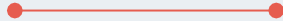
```
1  #include <Wire.h>
2  #include "bsec2.h"
3
```



```
3
4  Bsec2 envSensor;
5
```

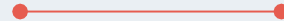
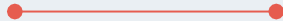
Setup

```
6  void setup() {  
7      Serial.begin(115200);  
8      Wire.begin(); // SDA to (GPIO21), SCL to (GPIO22)  
9  }
```



Initializing BME688

```
10   if (!envSensor.begin(BME68X_I2C_ADDR_LOW, Wire)) {  
11       Serial.println("BME688 not found. Check wiring.");  
12       while (1);  
13   }
```

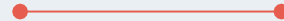
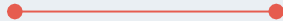


List of parameter

```
// Define which BSEC virtual sensors to use
bsec_virtual_sensor_t sensorList[] = {
    BSEC_OUTPUT_RAW_TEMPERATURE,
    BSEC_OUTPUT_RAW_PRESSURE,
    BSEC_OUTPUT_RAW_HUMIDITY,
    BSEC_OUTPUT_RAW_GAS,
    BSEC_OUTPUT_IAQ,
    BSEC_OUTPUT_CO2_EQUIVALENT,
    BSEC_OUTPUT_BREATH_VOC_EQUIVALENT
};
```

Pointer

```
37  const bsecOutputs* outputs = envSensor.getOutputs();  
38  // Get the pointer to the bsecOutputs struct
```



For loop printing

```
for (uint8_t i = 0; i < outputs->nOutputs; i++) {
    const bsec_output_t output = outputs->output[i];

    switch (output.sensor_id) {
        case BSEC_OUTPUT_RAW_TEMPERATURE:
            Serial.printf("Temperature (°C): %.2f\n", output.signal);
            break;
        case BSEC_OUTPUT_RAW_HUMIDITY:
            Serial.printf("Humidity (%): %.2f\n", output.signal);
            break;
        case BSEC_OUTPUT_RAW_PRESSURE:
            Serial.printf("Pressure (hPa): %.2f\n", output.signal / 100.0);
            break;
        case BSEC_OUTPUT_RAW_GAS:
            Serial.printf("Gas Resistance (Ohms): %.2f\n", output.signal);
            break;
        case BSEC_OUTPUT_IAQ:
            Serial.printf("IAQ: %.2f (Accuracy: %d)\n", output.signal, output.accuracy);
            break;
        case BSEC_OUTPUT_CO2_EQUIVALENT:
            Serial.printf("CO2 Equivalent (ppm): %.2f\n", output.signal);
            break;
        case BSEC_OUTPUT_BREATH_VOC_EQUIVALENT:
            Serial.printf("VOC Equivalent (ppm): %.2f\n", output.signal);
            break;
        default:
            Serial.printf("Unknown sensor ID: %d, value: %.2f\n", output.sensor_id, output.signal);
            break;
    }
}
```

Serial

```
02:16:40.030 -> ---- Sensor Readings ----
02:16:40.030 -> IAQ: 50.00 (Accuracy: 0)
02:16:40.030 -> CO2 Equivalent (ppm): 500.00
02:16:40.030 -> VOC Equivalent (ppm): 0.50
02:16:40.030 -> Temperature (° C): 22.71
02:16:40.030 -> Pressure (hPa): 10.16
02:16:40.030 -> Humidity (%): 57.30
02:16:40.030 -> Gas Resistance (Ohms): 163471.00
02:16:40.030 -> -----
```

```
02:21:03.682 -> ---- Sensor Readings ----
02:21:03.682 -> IAQ: 46.55 (Accuracy: 1)
02:21:03.682 -> CO2 Equivalent (ppm): 492.46
02:21:03.682 -> VOC Equivalent (ppm): 0.48
02:21:03.683 -> Temperature (° C): 22.76
02:21:03.683 -> Pressure (hPa): 10.16
02:21:03.683 -> Humidity (%): 56.59
02:21:03.683 -> Gas Resistance (Ohms): 102496.92
02:21:03.683 -> -----
```

```
00:45:04.981 -> ---- Sensor Readings ----
00:45:04.981 -> IAQ: 277.22 (Accuracy: 2)
00:45:05.023 -> CO2 Equivalent (ppm): 3144.21
00:45:05.023 -> VOC Equivalent (ppm): 200.84
00:45:05.023 -> Temperature (° C): 21.51
00:45:05.023 -> Pressure (hPa): 10.17
00:45:05.023 -> Humidity (%): 80.82
00:45:05.023 -> Gas Resistance (Ohms): 50954.42
00:45:05.023 -> -----
```

```
01:13:38.673 -> ---- Sensor Readings ----
01:13:38.673 -> IAQ: 136.40 (Accuracy: 3)
01:13:38.673 -> CO2 Equivalent (ppm): 1705.79
01:13:38.673 -> VOC Equivalent (ppm): 7.70
01:13:38.673 -> Temperature (° C): 21.18
01:13:38.705 -> Pressure (hPa): 10.16
01:13:38.705 -> Humidity (%): 80.43
01:13:38.705 -> Gas Resistance (Ohms): 75269.84
01:13:38.705 -> -----
```

Extracting data from Serial

```
D: > import serial.py > ...
1  import serial
2  import pandas as pd
3  from datetime import datetime
4
5  try:
6      ser = serial.Serial('COM17', 115200, timeout=2)
7      if not ser.is_open:
8          ser.open()
9  except serial.SerialException as e:
10     print(f"串口打开失败: {e}")
11     exit(1)
12
13     data = []
14     columns = ["VOC", "Gas", "Temp", "Humidity", "Label"]
15
16     print("Logging... Press Ctrl+C to stop.")
17
18     try:
19         while True:
20             line = ser.readline().decode('utf-8').strip()
21             print(line)
22             parts = line.split(',')
23             if len(parts) == 5:
24                 data.append(parts)
25
26     return _get_backend_mod().show(*args, **kwargs)
27     File "D:\代码\test\lib\site-packages\matplotlib\backend_bases.py", line 3553, in show
28         cls.mainloop()
29     File "D:\代码\test\lib\site-packages\matplotlib\backends\_backend_tk.py", line 520, in start_main_loop
30         first_manager.window.mainloop()
31     File "D:\lib\tkinter\__init__.py", line 1422, in mainloop
32         self.tk.mainloop(n)
33     KeyboardInterrupt
34     PS D:\代码\test> 
```

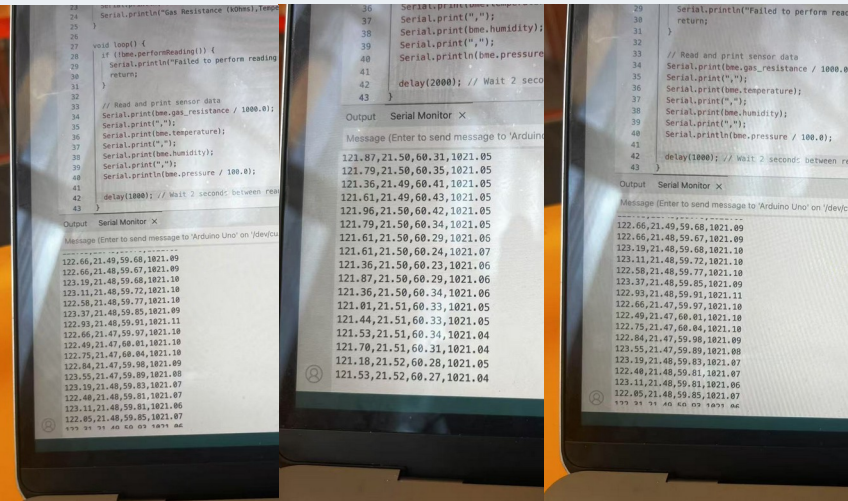
● ● ● ● ● ● ●

Response	Percentage
Yes	75%
No	25%

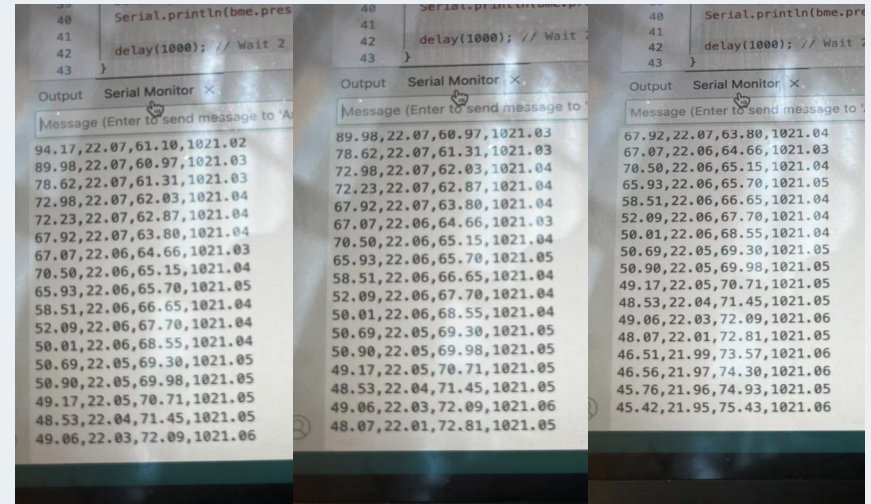




Machine Learning



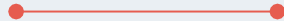
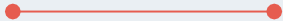
Manually collecting and training for Clean Air
Gas Resistance ~ 120 Ω



Manually collecting and training for Coffee
Gas Resistance ~ 50-60 Ω

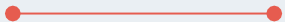
Unsupervised ML

```
14 # Select features for clustering
15 features = ['IAQ', 'VOC Equivalent (ppm)', 'Gas Resistance (Ohms)']
16 X = data[features]
17
18 # Normalize the features
19 scaler = StandardScaler()
20 X_scaled = scaler.fit_transform(X)
```

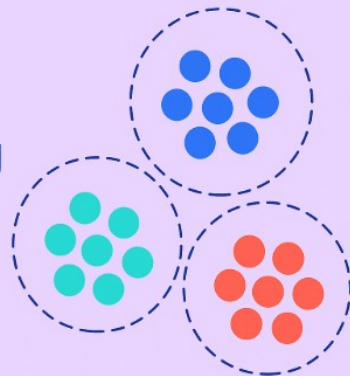


Unsupervised ML

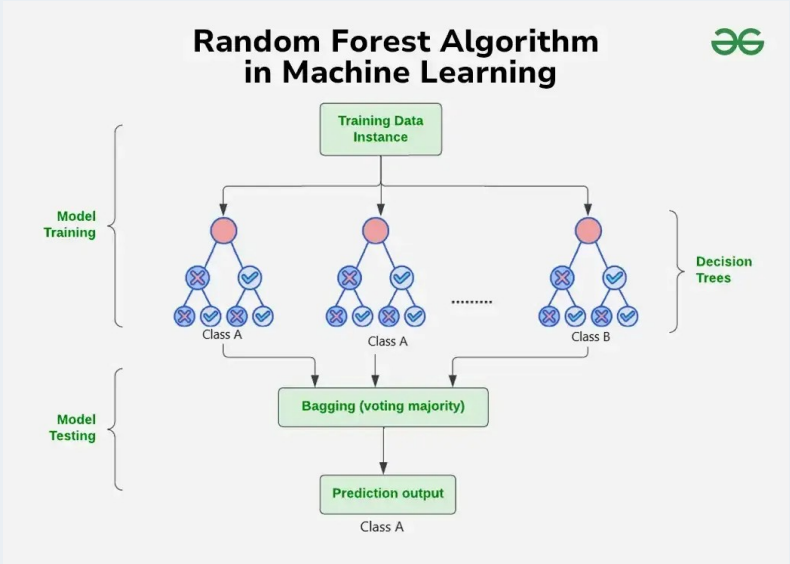
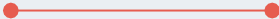
```
22 # Apply K-Means clustering (assuming 3 clusters for demonstration)
23 kmeans = KMeans(n_clusters=3, random_state=42)
24 data['Cluster'] = kmeans.fit_predict(X_scaled)
25
```



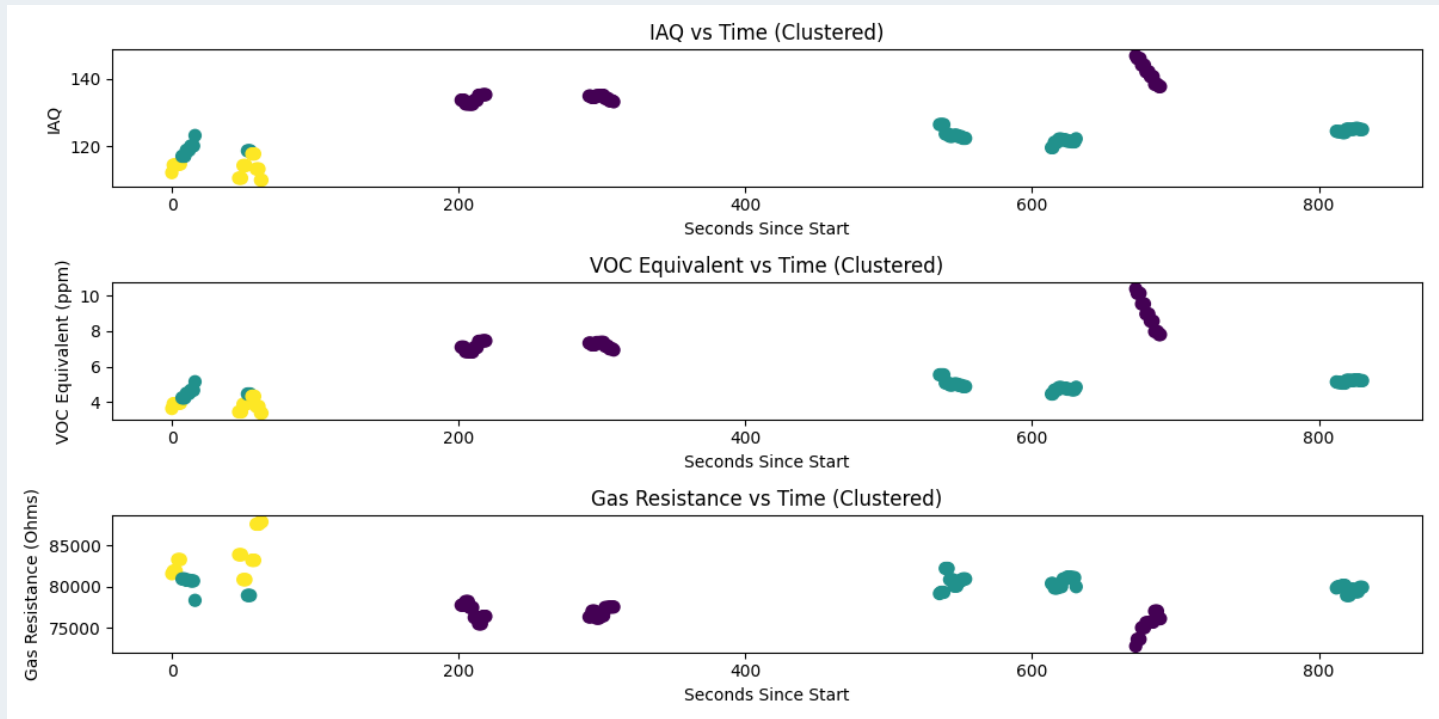
What Is
K-Means
Clustering



Why not supervised?



Research Results



Research Results



This video is a demonstration of the experimental results



Conclusion

Our sensor can differentiate between certain gases and smells. In the future we look forward to expanding our capabilities, integrating AI features, and overall improvement.

Our goal is to truly create something that can impact the world in a way for the better. To help people, to benefit the world.

